

Guia de deploy

Colocar a DevShowcase API no ar, de graça

São duas contas gratuitas: o **Supabase** guarda o banco de dados PostgreSQL, e o **Render** publica a API. Nenhuma das duas pede cartão de crédito. Leva mais ou menos 30 minutos.

Faça na ordem: primeiro o banco (Parte 1), depois a API (Parte 2). A Parte 2 precisa de uma informação que só aparece na Parte 1.

Parte 1 – Criar o banco no Supabase

Passo 1. Criar a conta

Entre em **supabase.com** e clique em **Start your project**. Pode entrar com a conta do GitHub (a mesma da Tatilane) – é o mais rápido, porque já vai estar logada.

Passo 2. Criar o projeto

Clique em **New project** e preencha:

- **Name:** devshowcase
- **Database Password:** clique em **Generate a password** e **copie essa senha para um bloco de notas**. Você vai precisar dela no Passo 3.
- **Region:** deixe a que vier, ou escolha South America (São Paulo).

Clique em **Create new project** e espere. Demora uns 2 minutos enquanto o banco é criado.

ATENÇÃO: essa senha não aparece de novo depois. Se perder, dá para gerar outra em **Settings → Database → Reset database password**.

Passo 3. Copiar o endereço do banco

Com o projeto criado, clique no botão **Connect** (fica no topo da página).

Procure a opção **Session pooler** e copie o endereço que aparece embaixo dela. Ele começa com **postgresql://** e é parecido com isto:

```
postgresql://postgres.abcdefgh:[YOUR-PASSWORD]@aws-0-sa-east-1.pooler.supabase.com:5432/postgres
```

No lugar de **[YOUR-PASSWORD]** (incluindo os colchetes), escreva a senha que você copiou no Passo 2. O endereço final fica assim:

```
postgresql://postgres.abcdefgh:SuaSenhaAqui@aws-0-sa-east-1.pooler.supabase.com:5432/postgres
```

Guarde esse endereço completo no bloco de notas. Ele é a **DATABASE_URL** e será usado no Passo 6.

Use o **Session pooler**, não o Direct connection. O Render não consegue se conectar pelo Direct connection.

Essa senha é uma credencial: não coloquem em nenhum arquivo do projeto, não mandem para o GitHub e não mostrem na tela durante o vídeo.

Parte 2 – Publicar a API no Render

Passo 4. Criar a conta

Entre em **render.com** e clique em **Get Started**. Escolha entrar com o **GitHub**, usando a conta da Tatilane (a mesma dona do repositório).

Passo 5. Criar o serviço da API

No painel, clique em **Add new** → **Web Service**.

Vai aparecer a lista dos repositórios do GitHub. Procure **devshowcase-api** e clique em **Connect**.

Se o repositório não aparecer, clique em **Configure account** e autorize o Render a ver os repositórios.

O projeto já tem um arquivo **render.yaml**, então o Render preenche quase tudo sozinho.

Confira se ficou assim:

- **Name:** devshowcase-api
- **Branch:** main
- **Build Command:** pip install -r requirements.txt
- **Start Command:** uvicorn app.main:app --host 0.0.0.0 --port \$PORT
- **Instance Type:** Free

Passo 6. Colocar a senha do banco

Ainda na mesma tela, procure **Environment Variables** (ou **Advanced** → **Add Environment Variable**) e adicione:

- **Key (nome):** DATABASE_URL
- **Value (valor):** o endereço completo que você guardou no Passo 3

É assim que as credenciais ficam fora do código: o endereço do banco e a senha ficam só aqui, no painel do Render.

Passo 7. Publicar

Clique em **Deploy Web Service** e espere. A primeira publicação demora de 3 a 5 minutos. Vocês vão ver o texto do build rolando na tela.

Deu certo quando aparecer **Live** em verde no topo da página, e no final do log aparecer **Application startup complete**.

Parte 3 – Conferir se funcionou

No topo da página do Render aparece o endereço público da API. É parecido com:

```
https://devshowcase-api.onrender.com
```

Esse é o link da API em produção que vai no PDF de entrega. Copie e guarde.

Abra no navegador, um de cada vez:

- **o endereço sozinho** → tem que aparecer a mensagem "DevShowcase API está funcionando"
- **o endereço + /docs** → abre o Swagger com todos os endpoints
- **o endereço + /api/technologies** → tem que aparecer uma lista vazia: []

A lista vazia é o certo: o banco na nuvem começa sem nada. Os cadastros do computador de vocês não vão para lá.

Agora é só usar esse endereço no Postman no lugar de `http://127.0.0.1:8000`, e cadastrar os dados da demonstração direto na nuvem.

Deploy contínuo (já vem ligado)

A partir de agora, toda vez que o código mudar no GitHub na branch main, o Render publica a versão nova sozinho. Não precisa mexer em mais nada. Dá para ver isso na aba **Events** do Render.

Se der errado

O que aparece	O que fazer
No Render: Build failed	Abra o log e procure a linha em vermelho. Quase sempre é o <code>requirements.txt</code> . Confira se o arquivo foi enviado para o GitHub.
No Render: Deploy failed com erro de conexão ao banco	A <code>DATABASE_URL</code> está errada. Confira se trocou <code>[YOUR-PASSWORD]</code> pela senha de verdade e se usou o Session pooler (não o Direct connection).
A primeira visita demora muito	Normal no plano gratuito: a API dorme depois de 15 minutos parada e leva uns 50 segundos para acordar. Antes de gravar o vídeo, abra o endereço e espere carregar.
password authentication failed	A senha do banco está errada. Gere outra em Settings → Database → Reset database password e atualize a <code>DATABASE_URL</code> no Render.
Mudei o código e a API não mudou	Confira se o commit foi enviado com git push. Veja a aba Events do Render.